



دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

دانشکده علوم ریاضی

پروژه سوم درس نرم افزار ریاضی

عنوان

درونیابی، تحلیل زمان اجرای الگوریتم‌ها و انتگرال گیری عددی

نگارش

ملیکا زارع

استاد درس

استاد حسینقلی زاده

تیر ۱۴۰۴

فصل ۱

مقدمه

در این پروژه سه مسئله‌ی اصلی با Python پیاده‌سازی شده و نتایج هر بخش به همراه کد، خروجی و نمودار گزارش می‌شود.

خلاصه‌ی کارهای انجام‌شده:

۱. بخش ۱: درونیابی نقاط (1,1) تا (6,2) با لاگرانژ و اسپلاین و استخراج ضابطه‌ی دقیق
۲. بخش ۲: خواندن خودکار Excel، رسم نمودارهای میله‌ای/خطی/جعبه‌ای، افزودن 700KB و محاسبه‌ی میانگین Alg.2
۳. بخش ۳: انتگرال‌گیری $f(x) = e^{x^2}$ روی (۰, ۱) با دوزنقه‌ای، سیمسون و گاوس
۴. بخش ۵: قرار دادن کدها در Repository عمومی GitHub

۱.۱ فایل‌های Python

- | | |
|---------------------------|---------------------------|
| بخش ۱ | • lagrange_and_spline.py |
| بخش ۲ | • runtime_from_excel.py |
| بخش ۳ | • integrate_exp_square.py |
| اجرای یک‌جای همه‌ی بخش‌ها | • main.py |

Repository ۱.۲

github.com/melikazre/numerical-analysis

فصل ۲

بخش ۱: درونیایی لاگرانژ و اسپلاین

۲.۱ شرح مسئله

طبق بخش ۱ پروژه، نقاط زیر در نظر گرفته شده‌اند:

$$(1, 1), (2, 3), (3, 5), (4, 8), (5, 5), (6, 2)$$

باید این داده‌ها با درونیایی لاگرانژ و اسپلاین درونیایی شوند و ضابطه‌ی دقیق هر روش به دست آید.

۲.۲ روش پیاده‌سازی

در لاگرانژ، چند جمله‌ای درجه‌ی ۵ ساخته می‌شود:

$$L(x) = \sum_{i=0}^{n-1} y_i \ell_i(x), \quad \ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{x - x_j}{x_i - x_j} \quad (2.1)$$

در اسپلاین مکعبی طبیعی، در هر بازه‌ی $[x_i, x_{i+1}]$:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad (2.2)$$

۲.۳ کد Python

۱.۲ برنامه - `lagrange_and_spline.py`

```
1 import numpy as np
```

```

۲ import matplotlib.pyplot as plt
۳ from scipy.interpolate import CubicSpline, lagrange
۴ import sympy as sp
۵ from pathlib import Path
۶
۷ results = Path(__file__).parent / "results"
۸ results.mkdir(exist_ok=True)
۹
۱۰ x_data = np.array([1, 2, 3, 4, 5, 6], dtype=float)
۱۱ y_data = np.array([1, 3, 5, 8, 5, 2], dtype=float)
۱۲
۱۳
۱۴ def get_lagrange_formula():
۱۵     x = sp.Symbol("x")
۱۶     n = len(x_data)
۱۷     polynomial = 0
۱۸
۱۹     for i in range(n):
۲۰         basis = y_data[i]
۲۱         for j in range(n):
۲۲             if i != j:
۲۳                 basis *= (x - x_data[j]) / (x_data[i] - x_data[j])
۲۴         polynomial += basis
۲۵
۲۶     return sp.expand(polynomial)
۲۷
۲۸
۲۹ def show_spline_pieces():
۳۰     spline = CubicSpline(x_data, y_data, bc_type="natural")
۳۱     pieces = []
۳۲
۳۳     for i in range(len(x_data) - 1):
۳۴         a, b, c, d = spline.c[:, i]
۳۵         x_start = x_data[i]
۳۶         text = (
۳۷             f"S{i + 1}(x) = {a:.4f} + {b:.4f}(x-{x_start})"
۳۸             f" + {c:.4f}(x-{x_start})^2 + {d:.4f}(x-{x_start})^3"
۳۹         )
۴۰         pieces.append((x_start, x_data[i + 1], text))
۴۱
۴۲     return pieces
۴۳
۴۴
۴۵ def draw_plot():
۴۶     x_smooth = np.linspace(x_data[0], x_data[-1], 300)

```

```

۴۷
۴۸     lagrange_curve = lagrange(x_data, y_data)(x_smooth)
۴۹     spline_curve = CubicSpline(x_data, y_data, bc_type="natural")(x_smooth)
۵۰
۵۱     plt.figure(figsize=(10, 6))
۵۲     plt.plot(x_smooth, lagrange_curve, label="Lagrange", linewidth=2)
۵۳     plt.plot(x_smooth, spline_curve, "--", label="Cubic spline", linewidth=2)
۵۴     plt.scatter(x_data, y_data, color="crimson", s=70, zorder=5, label="Data points")
۵۵     plt.xlabel("x")
۵۶     plt.ylabel("y")
۵۷     plt.title("Lagrange vs cubic spline interpolation")
۵۸     plt.legend()
۵۹     plt.grid(True, alpha=0.3)
۶۰     plt.tight_layout()
۶۱
۶۲     chart_path = results / "interpolation_plot.png"
۶۳     plt.savefig(chart_path, dpi=150)
۶۴     plt.close()
۶۵     return chart_path
۶۶
۶۷
۶۸ if __name__ == "__main__":
۶۹     print("Part 1 - Lagrange and spline interpolation\n")
۷۰
۷۱     print("Data points:")
۷۲     for x, y in zip(x_data, y_data):
۷۳         print(f"    ({int(x)}, {int(y)})")
۷۴
۷۵     formula = get_lagrange_formula()
۷۶     print("\nLagrange formula:")
۷۷     print(f"    L(x) = {formula}")
۷۸
۷۹     print("\nSpline pieces:")
۸۰     for start, end, piece in show_spline_pieces():
۸۱         print(f"    interval [{int(start)}, {int(end)}]: {piece}")
۸۲
۸۳     saved = draw_plot()
۸۴     print(f"\nPlot saved to: {saved}")

```

۲.۴ خروجی برنامه

خروجی اجرای بخش ۱ - برنامه ۲.۲

Part 1 - Lagrange and spline interpolation

Data points:

(1, 1) (2, 3) (3, 5) (4, 8) (5, 5) (6, 2)

Lagrange formula:

$L(x) = 0.175x^5 - 2.958333333333333x^4$
 $+ 18.375x^3 - 52.04166666666667x^2$
 $+ 68.45x - 31.0$

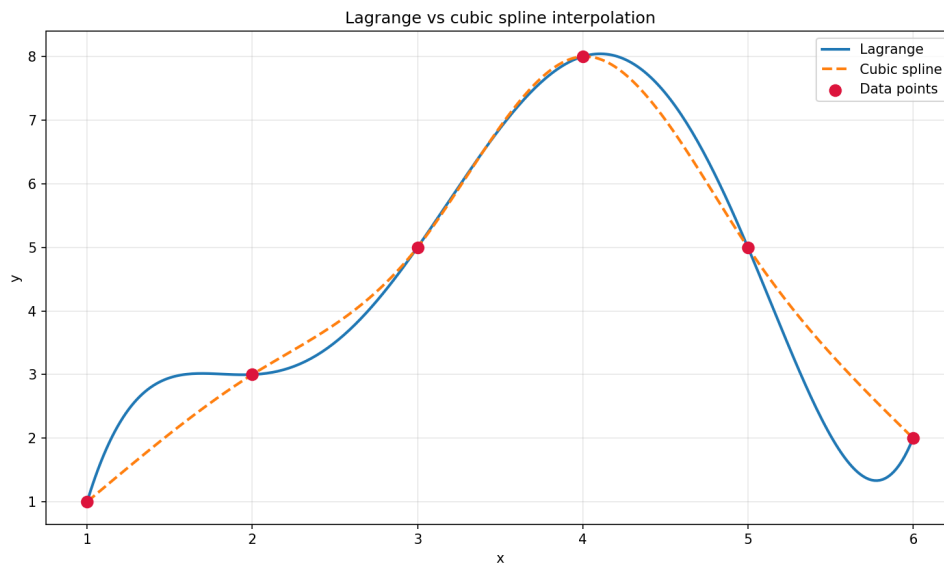
Spline pieces:

interval [1, 2]: $S_1(x) = -0.1866 + 0.0000(x-1.0)$
 $+ 2.1866(x-1.0)^2 + 1.0000(x-1.0)^3$
 interval [2, 3]: $S_2(x) = 0.9330 + -0.5598(x-2.0)$
 $+ 1.6268(x-2.0)^2 + 3.0000(x-2.0)^3$
 interval [3, 4]: $S_3(x) = -2.5455 + 2.2392(x-3.0)$
 $+ 3.3062(x-3.0)^2 + 5.0000(x-3.0)^3$
 interval [4, 5]: $S_4(x) = 2.2488 + -5.3971(x-4.0)$
 $+ 0.1483(x-4.0)^2 + 8.0000(x-4.0)^3$
 interval [5, 6]: $S_5(x) = -0.4498 + 1.3493(x-5.0)$
 $+ -3.8995(x-5.0)^2 + 5.0000(x-5.0)^3$

۲.۵ ضابطه‌ی دقیق لاگرانژ

$$L(x) = 0.175x^5 - 2.958333x^4 + 18.375x^3 - 52.0416x^2 + 68.45x - 31$$

۲.۶ نمودار



شکل ۲.۱: مقایسه‌ی منحنی‌های لاگرانژ و اسپلاین

۲.۷ تحلیل

- هر دو روش از تمام شش نقطه می‌گذرند.
- لاگرانژ: در بازه‌ی $[4, 6]$ نوسان شدید دارد (Runge phenomenon).
- اسپلاین: منحنی هموارتر و پایدارتر است.
- برای این داده‌ها، اسپلاین برای نمایش و تفسیر مناسب‌تر است.

فصل ۳

بخش ۲: تحلیل زمان اجرا از Excel

۳.۱ شرح مسئله

فایل runtime_data.xlsx زمان اجرای Alg.1، Alg.2 و Alg.3 را برای اندازه‌های 100KB تا 600KB (و سطر 700KB) ذخیره کرده است.

۳.۲ کد Python

runtime_from_excel.py - برنامه ۱.۳

```
۱ import pandas as pd
۲ import matplotlib.pyplot as plt
۳ from pathlib import Path
۴
۵ here = Path(__file__).parent
۶ excel_path = here / "runtime_data.xlsx"
۷ results = here / "results"
۸ results.mkdir(exist_ok=True)
۹
۱۰
۱۱ def read_excel_file():
۱۲     table = pd.read_excel(excel_path)
۱۳     table.columns = [str(name).strip() for name in table.columns]
۱۴     table = table.dropna(how="all")
۱۵     table = table[table.iloc[:, 0].notna()]
۱۶     return table
۱۷
۱۸
۱۹ def draw_bar_chart(table, size_column, algorithm_columns):
۲۰     labels = table[size_column].astype(str)
```

```

۲۱     positions = range(len(labels))
۲۲     bar_width = 0.25
۲۳
۲۴     fig, ax = plt.subplots(figsize=(10, 6))
۲۵     for i, col in enumerate(algorithm_columns):
۲۶         shift = (i - len(algorithm_columns) / 2 + 0.5) * bar_width
۲۷         ax.bar(
۲۸             [pos + shift for pos in positions],
۲۹             table[col],
۳۰             bar_width,
۳۱             label=col,
۳۲         )
۳۳
۳۴     ax.set_xlabel("Data size")
۳۵     ax.set_ylabel("Run time")
۳۶     ax.set_title("Bar chart - algorithm run times")
۳۷     ax.set_xticks(list(positions))
۳۸     ax.set_xticklabels(labels, rotation=45)
۳۹     ax.legend()
۴۰     ax.grid(True, axis="y", alpha=0.3)
۴۱     fig.tight_layout()
۴۲     fig.savefig(results / "runtime_bar_chart.png", dpi=150)
۴۳     plt.close(fig)
۴۴
۴۵
۴۶ def draw_line_chart(table, size_column, algorithm_columns):
۴۷     labels = table[size_column].astype(str)
۴۸
۴۹     fig, ax = plt.subplots(figsize=(10, 6))
۵۰     for col in algorithm_columns:
۵۱         ax.plot(labels, table[col], marker="o", linewidth=2, label=col)
۵۲
۵۳     ax.set_xlabel("Data size")
۵۴     ax.set_ylabel("Run time")
۵۵     ax.set_title("Line chart - how run time grows")
۵۶     ax.legend()
۵۷     ax.grid(True, alpha=0.3)
۵۸     fig.tight_layout()
۵۹     fig.savefig(results / "runtime_line_chart.png", dpi=150)
۶۰     plt.close(fig)
۶۱
۶۲
۶۳ def draw_box_plot(table, algorithm_columns):
۶۴     values_per_algorithm = [table[col].dropna() for col in algorithm_columns]
۶۵

```

```

۶۶ fig, ax = plt.subplots(figsize=(8, 6))
۶۷ ax.boxplot(values_per_algorithm, labels=algorithm_columns)
۶۸ ax.set_xlabel("Algorithm")
۶۹ ax.set_ylabel("Run time")
۷۰ ax.set_title("Box plot - spread of run times")
۷۱ ax.grid(True, axis="y", alpha=0.3)
۷۲ fig.tight_layout()
۷۳ fig.savefig(results / "runtime_box_plot.png", dpi=150)
۷۴ plt.close(fig)
۷۵
۷۶
۷۷ def average_alg2_between_100_and_600(table, algorithm_columns):
۷۸     size_column = table.columns[0]
۷۹     alg2_name = next(col for col in algorithm_columns if "2" in str(col))
۸۰
۸۱     wanted_sizes = ["100KB", "200KB", "300KB", "400KB", "500KB", "600KB"]
۸۲     filtered = table[table[size_column].astype(str).isin(wanted_sizes)]
۸۳     times = filtered[alg2_name].tolist()
۸۴     mean_time = filtered[alg2_name].mean()
۸۵
۸۶     return alg2_name, times, mean_time
۸۷
۸۸
۸۹ if __name__ == "__main__":
۹۰     print("Part 2 - runtime analysis from Excel\n")
۹۱
۹۲     table = read_excel_file()
۹۳     size_column = table.columns[0]
۹۴     algorithm_columns = [col for col in table.columns[1:] if str(col).strip()]
۹۵
۹۶     print(f"File: {excel_path.name}")
۹۷     print(f"Rows: {len(table)}\n")
۹۸     print(table.to_string(index=False))
۹۹
۱۰۰     draw_bar_chart(table, size_column, algorithm_columns)
۱۰۱     draw_line_chart(table, size_column, algorithm_columns)
۱۰۲     draw_box_plot(table, algorithm_columns)
۱۰۳
۱۰۴     alg2_name, times, mean_time = average_alg2_between_100_and_600(
۱۰۵         table, algorithm_columns
۱۰۶     )
۱۰۷
۱۰۸     print(f"\nAverage {alg2_name} for 100-600 KB:")
۱۰۹     print(f"  values: {times}")
۱۱۰     print(f"  mean:   {mean_time:.2f}")

```

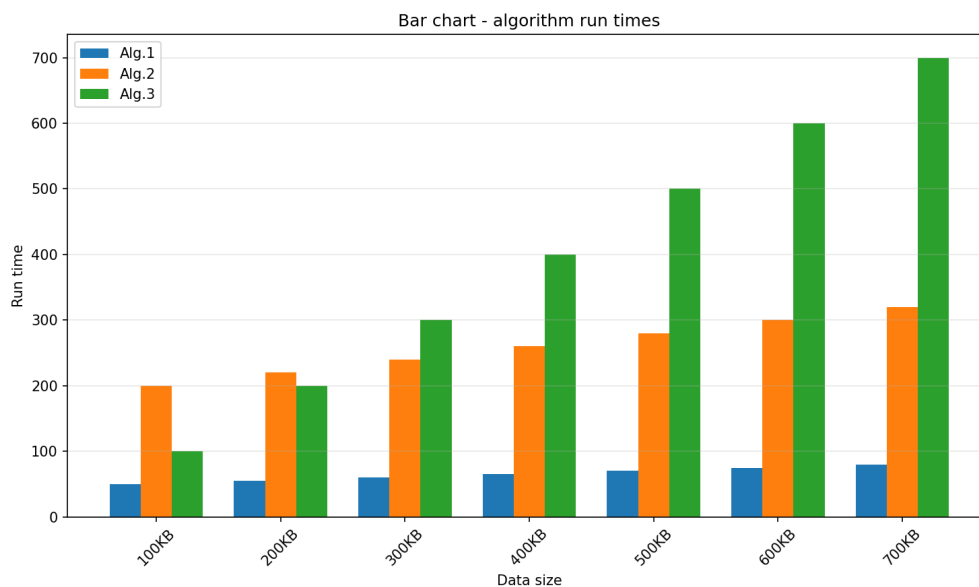
```
۱۱۱
۱۱۲     print("\nCharts saved:")
۱۱۳     print(f" {results / 'runtime_bar_chart.png'}")
۱۱۴     print(f" {results / 'runtime_line_chart.png'}")
۱۱۵     print(f" {results / 'runtime_box_plot.png'}")
```

۳.۳ بخش ۲-الف: خواندن Excel و رسم نمودار

کد با `pandas.read_excel` تمام سطرها و ستون‌ها را بدون ورود دستی می‌خواند و سه نمودار میله‌ای، خطی و جعبه‌ای رسم می‌کند.

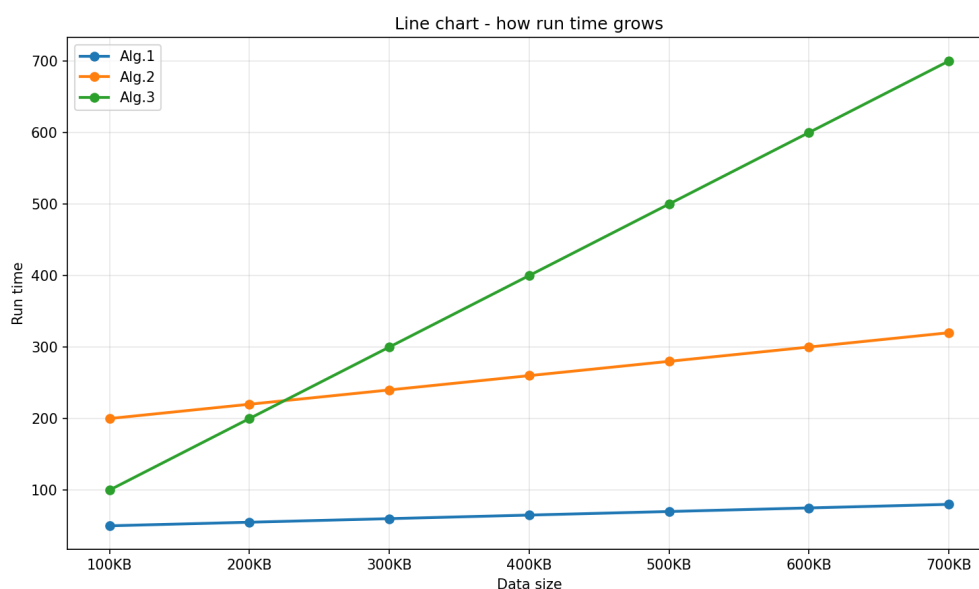
جدول ۳.۱: داده‌های خوانده‌شده از Excel

Data size	Alg.1	Alg.2	Alg.3
100KB	50	200	100
200KB	55	220	200
300KB	60	240	300
400KB	65	260	400
500KB	70	280	500
600KB	75	300	600
700KB	80	320	700



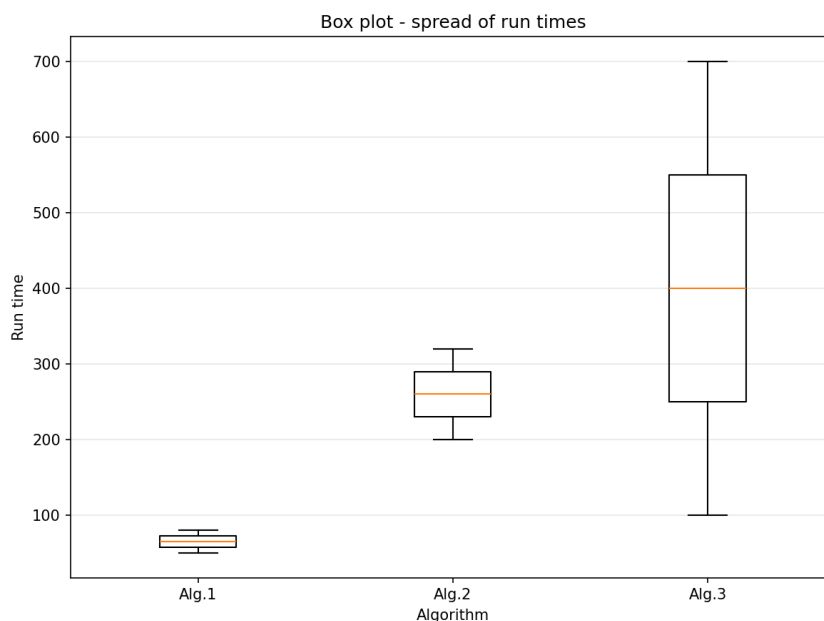
شکل ۳.۱: نمودار میله‌ای (بخش ۲-الف)

تحلیل نمودار میله‌ای: Alg.3 بیشترین زمان را دارد. Alg.1 سریع‌ترین الگوریتم است. با افزایش اندازه‌ی داده، زمان هر سه الگوریتم بیشتر می‌شود.



شکل ۳.۲: نمودار خطی (بخش ۲-الف)

تحلیل نمودار خطی: Alg.3 تقریباً خط راست با شیب ۱ است (زمان $\approx$ اندازه‌ی داده). Alg.2 رشد خطی با شیب کمتر دارد. Alg.1 کندترین رشد را دارد.



شکل ۳.۳: نمودار جعبه‌ای (بخش ۲-الف)

تحلیل نمودار جعبه‌ای: Alg.1 کم‌ترین پراکندگی (50-80) را دارد. Alg.3 بیشترین دامنه (100-700) و حساس‌ترین الگوریتم به اندازه‌ی داده است.

۳.۴ بخش ۲-ب: افزودن سطر 700KB

سطر زیر به Excel اضافه شده است:

700KB 80 320 700

با اجرای مجدد کد بدون هیچ تغییری، نمودارها به‌روز شدند و سطر 700KB در جدول و نمودارها دیده می‌شود.

۳.۵ بخش ۲-ج: میانگین Alg.2

خروجی میانگین Alg.2 - برنامه ۲.۳

Average Alg.2 for 100-600 KB:

```
values: [200, 220, 240, 260, 280, 300]  
mean:   250.00
```

$$\bar{t}_{\text{Alg}_2} = \frac{200 + 220 + 240 + 260 + 280 + 300}{6} = 250.00$$

میانگین زمان اجرای Alg.2 برای داده‌های 100KB تا 600KB برابر **250.00** است.

فصل ۴

بخش ۳: انتگرال گیری $f(x) = e^{x^2}$

۴.۱ شرح مسئله

تابع $f(x) = e^{x^2}$ در نظر گرفته شده و مقدار انتگرال روی $(0, 1)$ محاسبه می شود:

$$I = \int_0^1 e^{x^2} dx$$

۴.۲ کد Python

integrate_exp_square.py - برنامه ۱.۴

```
۱ import numpy as np
۲ from numpy.polynomial.legendre import leggauss
۳ from scipy import integrate
۴ from pathlib import Path
۵
۶ results = Path(__file__).parent / "results"
۷ results.mkdir(exist_ok=True)
۸
۹
۱۰ def f(x):
۱۱     return np.exp(x ** 2)
۱۲
۱۳
۱۴ def trapezoid_rule(start, end, pieces=1000):
۱۵     x = np.linspace(start, end, pieces + 1)
۱۶     y = f(x)
۱۷     step = (end - start) / pieces
۱۸     return step * (0.5 * y[0] + y[1:-1].sum() + 0.5 * y[-1])
۱۹
```



```

۲۰
۲۱ def simpson_rule(start, end, pieces=1000):
۲۲     if pieces % 2 == 1:
۲۳         pieces += 1
۲۴
۲۵     x = np.linspace(start, end, pieces + 1)
۲۶     y = f(x)
۲۷     step = (end - start) / pieces
۲۸     return step / 3 * (
۲۹         y[0] + y[-1] + 4 * y[1:-1:2].sum() + 2 * y[2:-2:2].sum()
۳۰     )
۳۱
۳۲
۳۳ def gauss_quadrature(start, end, points=10):
۳۴     nodes, weights = leggauss(points)
۳۵     mapped_x = 0.5 * (end - start) * nodes + 0.5 * (end + start)
۳۶     return 0.5 * (end - start) * (weights * f(mapped_x)).sum()
۳۷
۳۸
۳۹ if __name__ == "__main__":
۴۰     start, end = 0.0, 1.0
۴۱     pieces = 1000
۴۲
۴۳     print("Part 3 - integrating f(x) = e^(x^2) on (0, 1)\n")
۴۴
۴۵     by_trapezoid = trapezoid_rule(start, end, pieces)
۴۶     by_simpson = simpson_rule(start, end, pieces)
۴۷     by_gauss = gauss_quadrature(start, end, points=10)
۴۸     reference, _ = integrate.quad(f, start, end)
۴۹
۵۰     print(f"Trapezoidal (n={pieces}): {by_trapezoid:.10f}")
۵۱     print(f"Simpson (n={pieces}): {by_simpson:.10f}")
۵۲     print(f"Gauss (10 points): {by_gauss:.10f}")
۵۳     print(f"Reference (scipy): {reference:.10f}")
۵۴
۵۵     print("\nError vs reference:")
۵۶     print(f"   trapezoidal: {abs(by_trapezoid - reference):.2e}")
۵۷     print(f"   simpson: {abs(by_simpson - reference):.2e}")
۵۸     print(f"   gauss: {abs(by_gauss - reference):.2e}")
۵۹
۶۰     output_file = results / "integration_results.txt"
۶۱     with open(output_file, "w", encoding="utf-8") as file:
۶۲         file.write("Integral of e^(x^2) on [0, 1]\n")
۶۳         file.write("-" * 40 + "\n")
۶۴         file.write(f"Trapezoidal: {by_trapezoid:.10f}\n")

```

```

۶۵     file.write(f"Simpson:      {by_simpson:.10f}\n")
۶۶     file.write(f"Gauss:      {by_gauss:.10f}\n")
۶۷     file.write(f"Reference:   {reference:.10f}\n")
۶۸
۶۹     print(f"\nResults saved to: {output_file}")

```

۴.۳ بخش ۳-الف: دوزنقه‌ای و سیمسون

$$I \approx h \left[\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(x_i) \right] \quad (\text{دوزنقه‌ای}) \quad (4.1)$$

$$I \approx \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{\text{odd}} f(x_i) + 2 \sum_{\text{even}} f(x_i) \right] \quad (\text{سیمسون}) \quad (4.2)$$

۴.۴ بخش ۳-ب: گاوس-لژاندر و مقایسه

انتگرال با 10 نقطه‌ی گاوس روی $[-1, 1]$ محاسبه و به بازه‌ی $[0, 1]$ نگاشت شده است. نتایج با `scipy.integrate.quad` مقایسه شده‌اند.

جدول ۴.۱: مقایسه‌ی روش‌های انتگرال گیری (بخش ۳)

Method	Value	Error vs reference
Trapezoidal ($n = 1000$)	1.4626521990	4.53×10^{-7}
Simpson ($n = 1000$)	1.4626517459	3.02×10^{-13}
Gauss (10 points)	1.4626517459	6.66×10^{-16}
Reference (scipy)	1.4626517459	—

خروجی اجرای بخش ۳ - برنامه ۲.۴

```
Part 3 - integrating f(x) = e^(x^2) on (0, 1)
```

```
Trapezoidal (n=1000): 1.4626521990
```

```
Simpson (n=1000): 1.4626517459
```

```
Gauss (10 points): 1.4626517459
```

```
Reference (scipy): 1.4626517459
```

```
Error vs reference:
```

```
trapezoidal: 4.53e-07
```

```
simpson: 3.02e-13
```

```
gauss: 6.66e-16
```

۴.۵ تحلیل مقایسه‌ای

- دوزنقه‌ای: ساده است اما با $n = 1000$ هنوز خطای 4.53×10^{-7} دارد.
- سیمسون: با همان $n = 1000$ تقریباً با مرجع یکسان است (خطای 3.02×10^{-13}).
- گاوس: تنها با 10 ارزیابی تابع به دقت ماشین (10^{-16}) می‌رسد.
- نتیجه: برای e^{x^2} روی $[0, 1]$ ، سیمسون و گاوس از دوزنقه‌ای دقیق‌ترند؛ گاوس بهترین نسبت دقت به هزینه را دارد.

فصل ۵

نتیجه‌گیری

در این پروژه تمام موارد خواسته‌شده پیاده‌سازی و گزارش شد:

۱. بخش ۱: ضابطه‌ی لاگرانژ و پنج قطعه‌ی اسپلاین استخراج شد. اسپلاین هموارتر از لاگرانژ بود.
۲. بخش ۲: Excel خودکار خوانده شد؛ سه نمودار رسم و تحلیل شد. سطر 700KB بدون تغییر کد اضافه شد. میانگین Alg.2 برابر 250.00 شد.
۳. بخش ۳: $I \approx 1/4626517459$ محاسبه شد. سیمسون و گاوس دقیق‌تر از دوزنقه‌ای بودند.
۴. بخش ۵: کدها در Repository عمومی GitHub قرار گرفتند.